

23070521054- Gauri Gulhane

```
# =====  
# PRACTICAL 5  
# Hyperparameter Tuning and Model Optimization  
# =====  
  
import numpy as np  
import pandas as pd  
import tensorflow as tf  
  
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Input  
  
# -----  
# 1. Load Dataset  
# -----  
  
data = load_breast_cancer()  
  
X = data.data  
y = data.target  
  
print("Dataset Shape:", X.shape)  
  
# -----  
# 2. Split Dataset  
# -----  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)  
  
# -----  
# 3. Feature Scaling  
# -----  
  
scaler = StandardScaler()  
  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)  
  
# -----  
# 4. Hyperparameter Configurations
```

```

# -----
configurations = [

    {
        "layers": 1,
        "units": 32,
        "learning_rate": 0.001,
        "batch_size": 32,
        "optimizer": "adam"
    },

    {
        "layers": 2,
        "units": 64,
        "learning_rate": 0.001,
        "batch_size": 32,
        "optimizer": "adam"
    },

    {
        "layers": 2,
        "units": 64,
        "learning_rate": 0.0005,
        "batch_size": 64,
        "optimizer": "adam"
    },

    {
        "layers": 3,
        "units": 128,
        "learning_rate": 0.0005,
        "batch_size": 32,
        "optimizer": "adam"
    },

    {
        "layers": 2,
        "units": 64,
        "learning_rate": 0.01,
        "batch_size": 32,
        "optimizer": "sgd"
    },

    {
        "layers": 2,
        "units": 64,
        "learning_rate": 0.001,
        "batch_size": 32,
        "optimizer": "rmsprop"
    }
]

# -----
# 5. Model Creation Function
# -----

def create_model(config):

    model = Sequential()

    model.add(
        Input(

```

```

        shape=(X_train.shape[1],)
    )
)

for i in range(config["layers"]):

    model.add(
        Dense(
            config["units"],
            activation='relu'
        )
    )

model.add(
    Dense(
        1,
        activation='sigmoid'
    )
)

optimizer = tf.keras.optimizers.get({
    "class_name": config["optimizer"],
    "config": {"learning_rate": config["learning_rate"]}
})

model.compile(
    optimizer=optimizer,
    loss='binary_crossentropy',
    metrics=['accuracy']
)

return model

# -----
# 6. Test Hyperparameters
# -----

results = []
histories = [] # To store training histories

for i, config in enumerate(
    configurations
):

    print(
        "\nTesting Configuration",
        i + 1
    )

    print(config)

    model = create_model(config)

    history = model.fit(
        X_train,
        y_train,
        validation_split=0.20,
        epochs=25,
        batch_size=config[
            "batch_size"

```

```
    ],
    verbose=0
)

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

histories.append(history) # Store the history

results.append({

    "Layers":
        config["layers"],

    "Units":
        config["units"],

    "Learning Rate":
        config["learning_rate"],

    "Batch Size":
        config["batch_size"],
    "Optimizer":
        config["optimizer"],
    "Test Accuracy":
        accuracy,

    "Test Loss":
        loss
})

print(
    "Test Accuracy:",
    round(accuracy, 4)
)

# -----
# 7. Results
# -----

results_df = pd.DataFrame(
    results
)

print("\nHyperparameter Results:")
print(results_df)

# -----
# 8. Display Best Measured Configuration
# -----

best_index = results_df[
    "Test Accuracy"
].idxmax()

print(
```

```

    "\nConfiguration with Highest "
    "Measured Test Accuracy:"
)

print(
    results_df.loc[
        best_index
    ]
)

```

Dataset Shape: (569, 30)

Testing Configuration 1

```
{'layers': 1, 'units': 32, 'learning_rate': 0.001, 'batch_size': 32, 'optimizer': 'adam'}
```

Test Accuracy: 0.9561

Testing Configuration 2

```
{'layers': 2, 'units': 64, 'learning_rate': 0.001, 'batch_size': 32, 'optimizer': 'adam'}
```

Test Accuracy: 0.9649

Testing Configuration 3

```
{'layers': 2, 'units': 64, 'learning_rate': 0.0005, 'batch_size': 64, 'optimizer': 'adam'}
```

Test Accuracy: 0.9474

Testing Configuration 4

```
{'layers': 3, 'units': 128, 'learning_rate': 0.0005, 'batch_size': 32, 'optimizer': 'adam'}
```

Test Accuracy: 0.9474

Testing Configuration 5

```
{'layers': 2, 'units': 64, 'learning_rate': 0.01, 'batch_size': 32, 'optimizer': 'sgd'}
```

Test Accuracy: 0.9298

Testing Configuration 6

```
{'layers': 2, 'units': 64, 'learning_rate': 0.001, 'batch_size': 32, 'optimizer': 'rmsprop'}
```

Test Accuracy: 0.9737

Hyperparameter Results:

	Layers	Units	Learning Rate	Batch Size	Optimizer	Test Accuracy \
0	1	32	0.0010	32	adam	0.956140
1	2	64	0.0010	32	adam	0.964912
2	2	64	0.0005	64	adam	0.947368
3	3	128	0.0005	32	adam	0.947368
4	2	64	0.0100	32	sgd	0.929825
5	2	64	0.0010	32	rmsprop	0.973684

Test Loss

0	0.097018
1	0.100012
2	0.117743
3	0.118705
4	0.180877
5	0.095725

Configuration with Highest Measured Test Accuracy:

Layers	2
Units	64
Learning Rate	0.001
Batch Size	32
Optimizer	rmsprop
Test Accuracy	0.973684
Test Loss	0.095725
Name: 5, dtype: object	

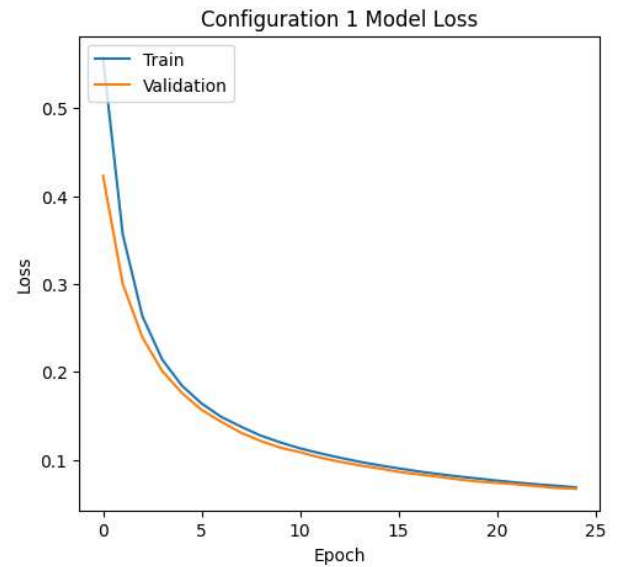
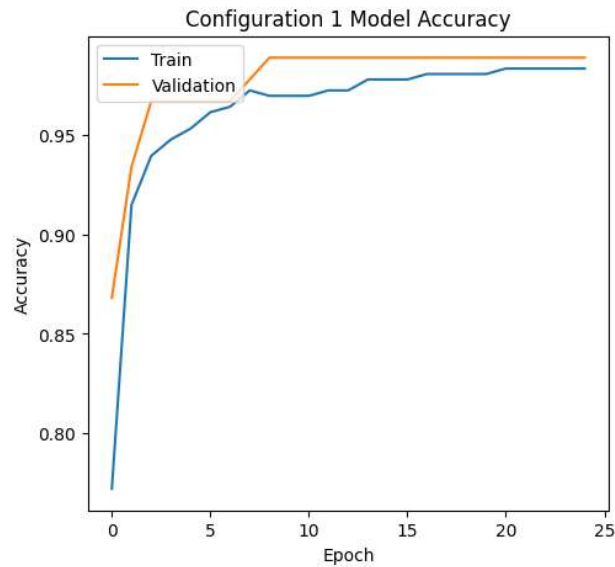
```
# -----
# 9. Visualize Training History (Accuracy and Loss)
# -----

for i, history in enumerate(histories):
    print(f"\nPlotting history for Configuration {i+1} ({configurations[i]['optimizer']} - LR: {c

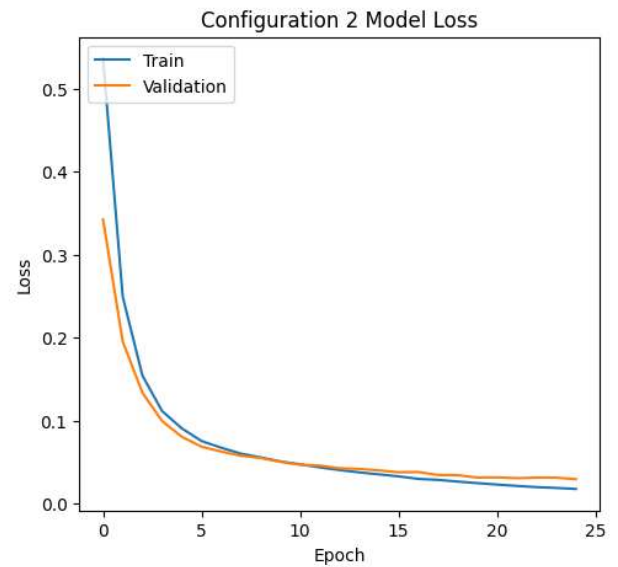
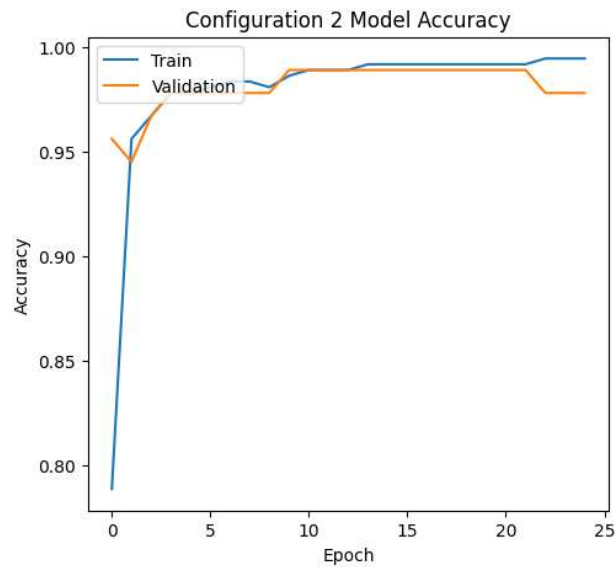
    # Plot training & validation accuracy values
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title(f'Configuration {i+1} Model Accuracy')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Validation'], loc='upper left')

    # Plot training & validation loss values
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title(f'Configuration {i+1} Model Loss')
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Validation'], loc='upper left')
    plt.show()
```

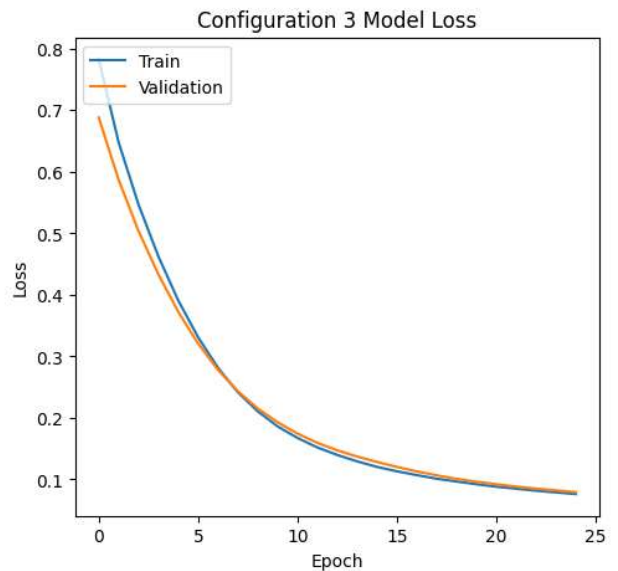
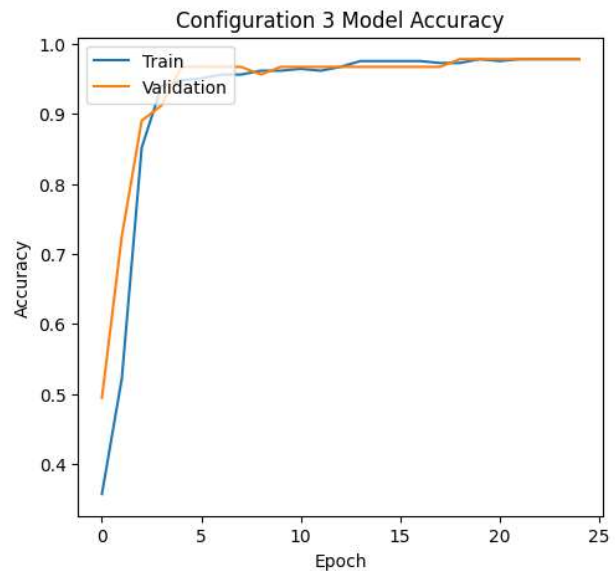

Plotting history for Configuration 1 (adam - LR: 0.001)



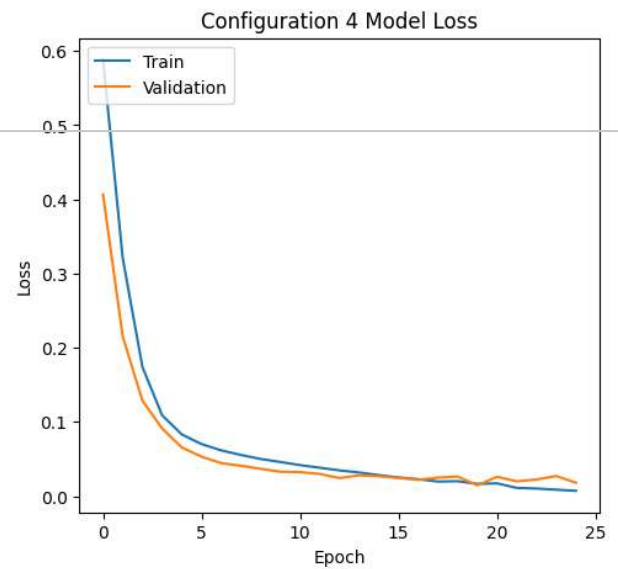
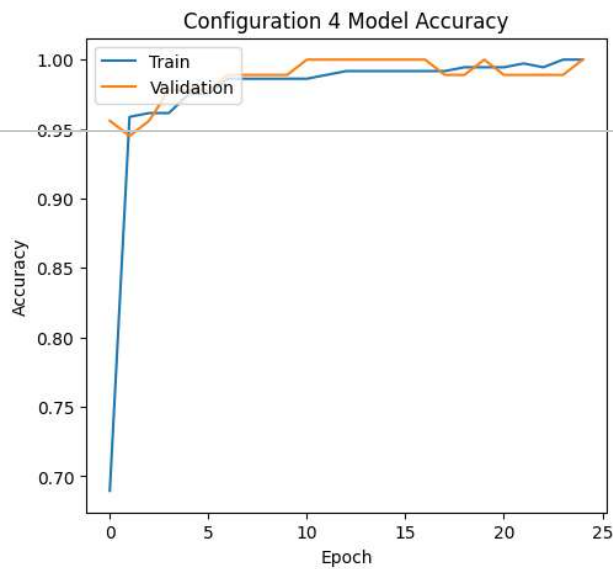
Plotting history for Configuration 2 (adam - LR: 0.001)



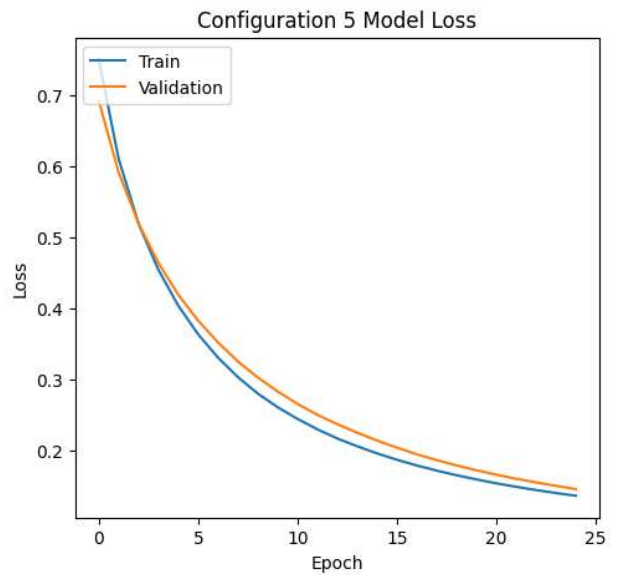
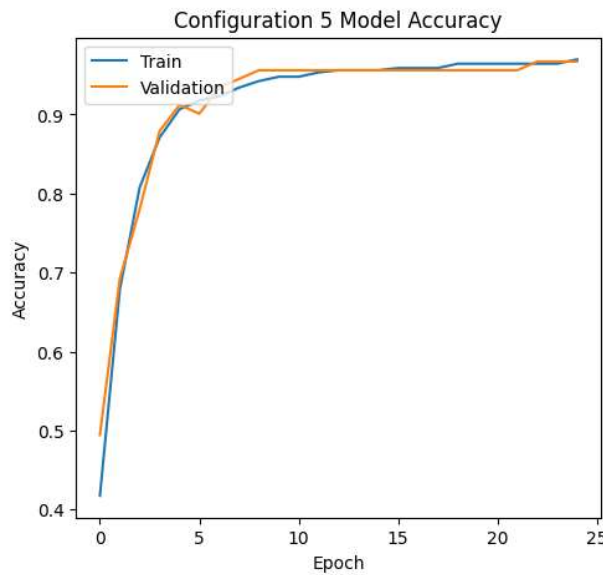
Plotting history for Configuration 3 (adam - LR: 0.0005)



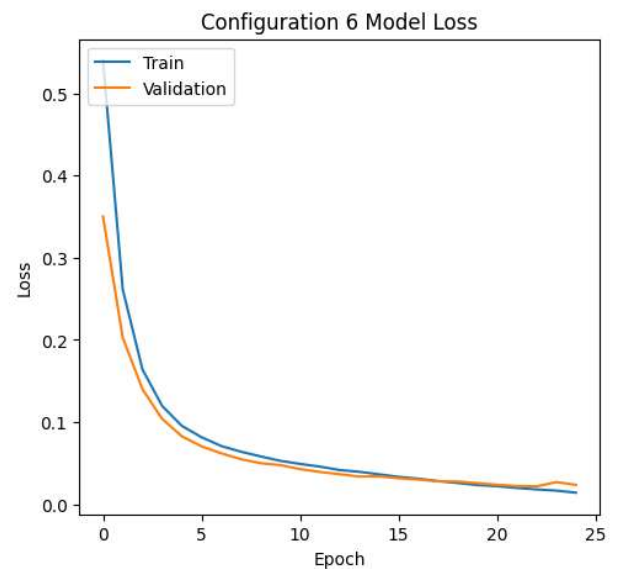
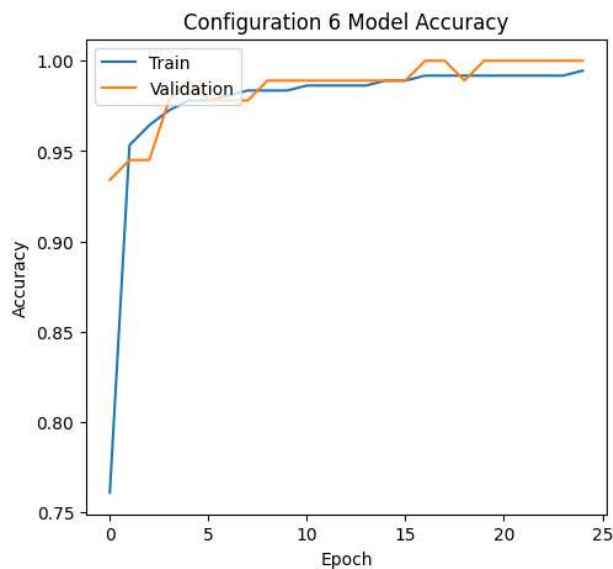
Plotting history for Configuration 4 (adam - LR: 0.0005)



Plotting history for Configuration 5 (sgd - LR: 0.01)



Plotting history for Configuration 6 (rmsprop - LR: 0.001)



```
# -----  
# 10. Generate Confusion Matrix for Best Model  
# -----  
  
# Retrain the best model to get its predictions  
best_config = results_df.loc[best_index]  
best_model_config = {  
    "layers": int(best_config["Layers"]),  
    "units": int(best_config["Units"]),  
    "learning_rate": float(best_config["Learning Rate"]),  
    "batch_size": int(best_config["Batch Size"]),  
    "optimizer": best_config["Optimizer"]  
}  
  
best_model = create_model(best_model_config)  
  
best_model.fit(  
    X_train,  
    y_train,  
    epochs=25, # Use the same number of epochs as during tuning  
    batch_size=best_model_config["batch_size"],  
    verbose=0  
)  
  
# Predict on the test set  
y_pred_prob = best_model.predict(X_test)  
y_pred = (y_pred_prob > 0.5).astype(int)  
  
# Generate confusion matrix  
cm = confusion_matrix(y_test, y_pred)  
  
# Display confusion matrix  
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=data.target_names)  
disp.plot(cmap=plt.cm.Blues)  
plt.title('Confusion Matrix for Best Model')  
plt.show()
```